

MAST32001 Week 1 Summary

Zhi Zhou

The lectures mixed “I get it” with “What have happened”. It started with what computational statistics is. It is not just plugging numbers into a formula, but designing and using computer methods for statistical problems.

We reviewed variance basics, then we compared three ways, two-pass, one-pass formula using sums of x and x^2 , and Welford’s method. For normal data, they all matched `var()`. But when the mean was huge, like 10^{12} , the one-pass formula returned 0. The problem is catastrophic cancellation. Subtracting two large, nearly identical numbers destroys all precision. First lesson: mathematically identical algorithms are not computationally identical.

Computers do not store real numbers exactly. In R, double precision only gives you about 15-16 digits, so tiny rounding errors can snowball. Machine epsilon, overflow, underflow, and even addition order matter. In practice, use log-likelihoods instead of multiplying probabilities, use `pnorm(..., lower.tail = FALSE, log.p = TRUE)` for tails, and apply the log-sum-exp trick. Second lesson: a correct formula is not necessarily numerically stable.

Least squares, PCA, covariance matrices, they all use to solve $Ax = b$. And do not solve A . It is slower, less accurate, and wastes memory. Solve the system directly instead.

We also talked about norms and the condition number $\kappa(A) = \sigma_{max}/\sigma_{min}$. A huge condition number means the problem is super sensitive to tiny input changes. In one exercise, a design matrix with a condition number ~3 million gave completely unstable coefficients when we changed one response value, even though the fitted values stayed stable. Third lesson: predictions can be fine while the individual coefficients are meaningless.

For decompositions, match the tool to the job: LU for general square systems, Cholesky for symmetric positive-definite, QR for least squares, and SVD for diagnostic rank. For least squares, never use $X^T X$ because it squares the condition number and makes everything more unstable. Use `qr.solve()` or `lm()`. In R, it is also better to use `corssprod()` instead of `t(x) %*% x`, avoid explicit inverses, center or scale your predictors, and check residuals.

The math was not too bad. The hard part is learning that a correct formula can quietly give wrong answers. In nearly collinear regression, coefficients can be all over the place even when the predictions look fine. It makes me think differently about computing: not just about algebra, but about how number actually insider a computer.